

API Security Risk Analysis Report

Target: ReqRes.in Demo API | Task 3 — Future Interns Cybersecurity Program

Prepared by: Priyadharshini S | Date: June 2026 | Classification: Confidential

API Tested	Total Risks	High Severity	Scope
ReqRes.in	7 Identified	3 Critical	Read-Only / Demo

1. Task Overview

This assessment was conducted as part of Future Interns Cybersecurity Program — Task 3: API Security Risk Analysis. The task required analyzing public/demo APIs to identify common security risks, assess authentication and access control, and document findings in a professional security report.

2. Executive Summary

This report presents the findings of a security risk analysis performed on the ReqRes.in public demo API. The assessment was conducted using Postman for endpoint testing and manual inspection of API responses, headers, and authentication behavior.

The analysis identified 7 distinct security risks across authentication, authorization, data exposure, and rate limiting categories. Three risks were classified as HIGH severity. All findings follow OWASP API Security Top 10 guidelines with actionable remediation recommendations.

3. Scope & Methodology

3.1 Target API

- Platform: ReqRes.in — a publicly available hosted API for testing and prototyping
- Base URL: <https://reqres.in>
- Testing Tool: Postman (API Security Audit collection)
- Testing Type: Read-only, non-exploitative, black-box analysis

3.2 Endpoints Tested

- GET /api/users — Retrieve all users (paginated)
- GET /api/users/{id} — Retrieve single user by ID (tested /2 and /5)
- POST /api/login — User authentication
- POST /api/register — New user registration

3.3 Methodology

- Documentation Review: Examined API docs for authentication requirements
- Endpoint Enumeration: Sent unauthenticated requests to all endpoints
- Authentication Testing: Tested with missing/invalid tokens
- Response Analysis: Inspected returned data for excessive exposure
- Risk Classification: Rated using OWASP API Top 10 severity scoring

4. Testing Evidence — Postman Screenshots

The following screenshots were captured during live testing of the ReqRes.in API using Postman. They provide direct evidence for the identified security findings.

4.1 Unauthenticated Access — GET /api/users/2 (401 Response)

When testing GET /api/users/2 without an API key, the API returned a 401 Unauthorized error with a JSON body explaining that the x-api-key header is required. This confirms the endpoint is protected — however, inconsistency was observed across endpoints (see Finding R-01).

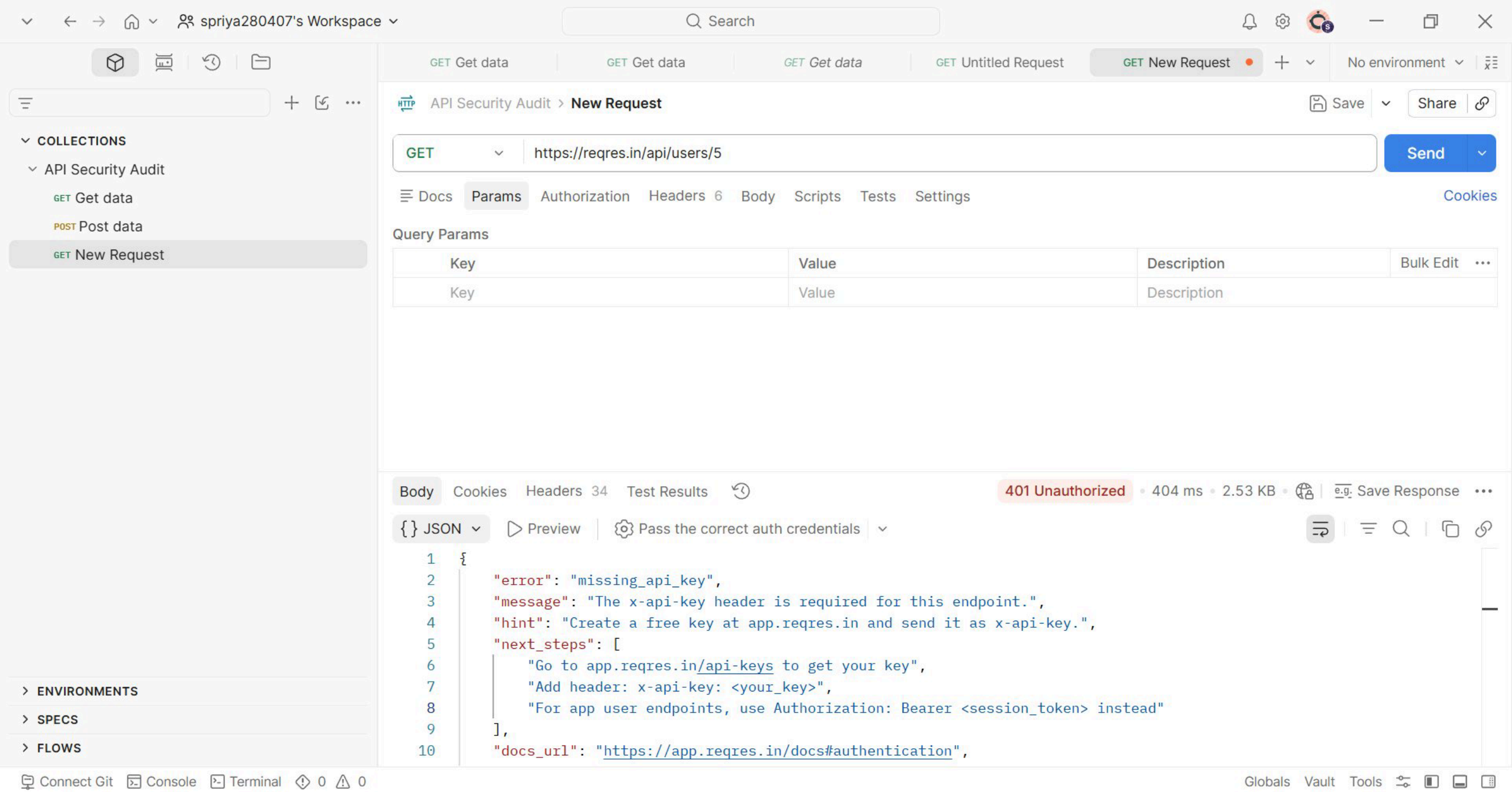


Figure 2: GET /api/users/2 – 401 Unauthorized response (missing x-api-key header)

4.2 IDOR Testing – GET /api/users/2 vs /api/users/5

By simply changing the integer ID in the URL path from /2 to /5, a different user's profile was returned. This demonstrates the Insecure Direct Object Reference (IDOR) vulnerability where sequential numeric IDs allow enumeration of all user records.

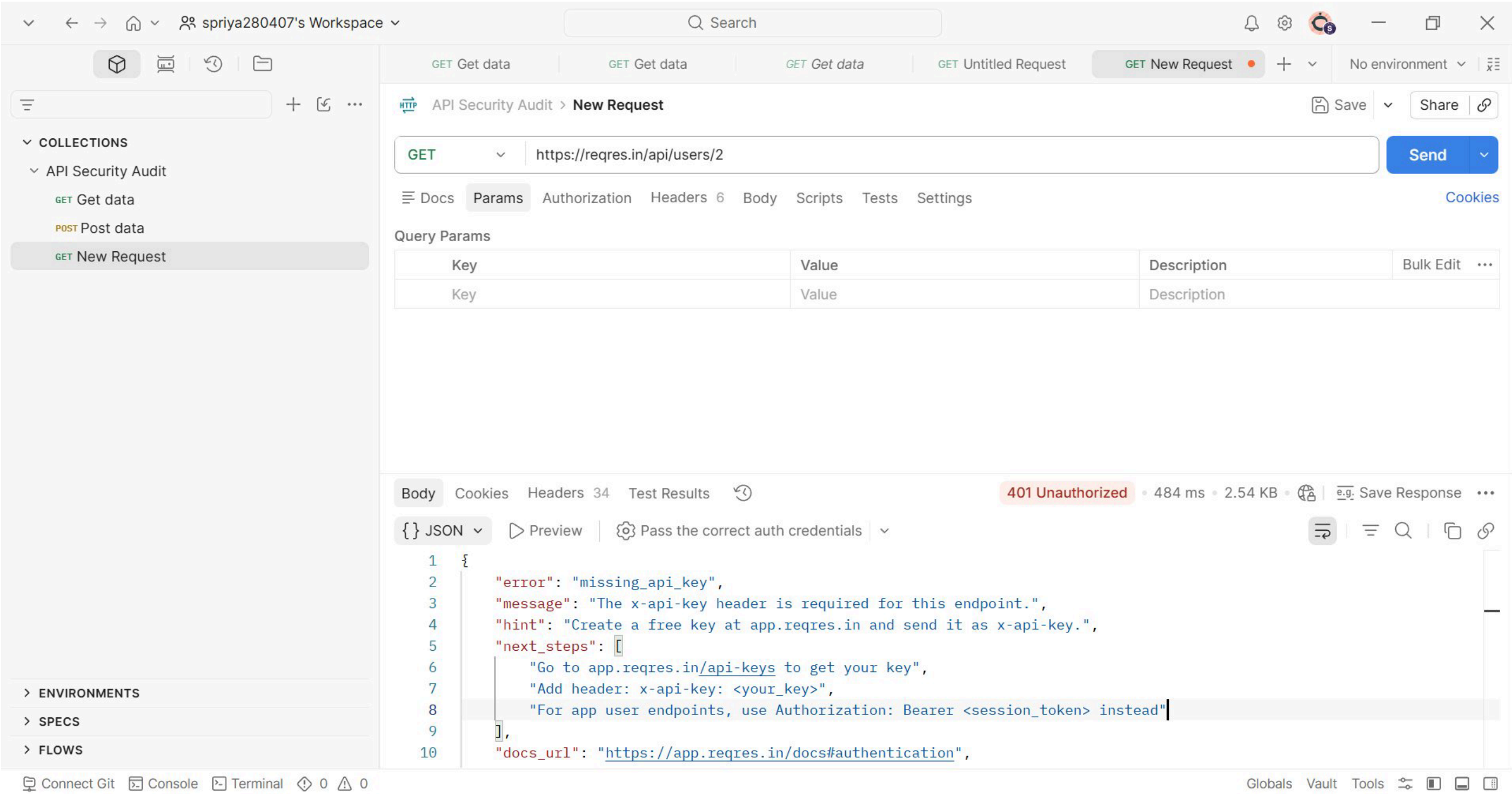


Figure 3: GET /api/users/2 – Response showing 401 with missing_api_key error detail

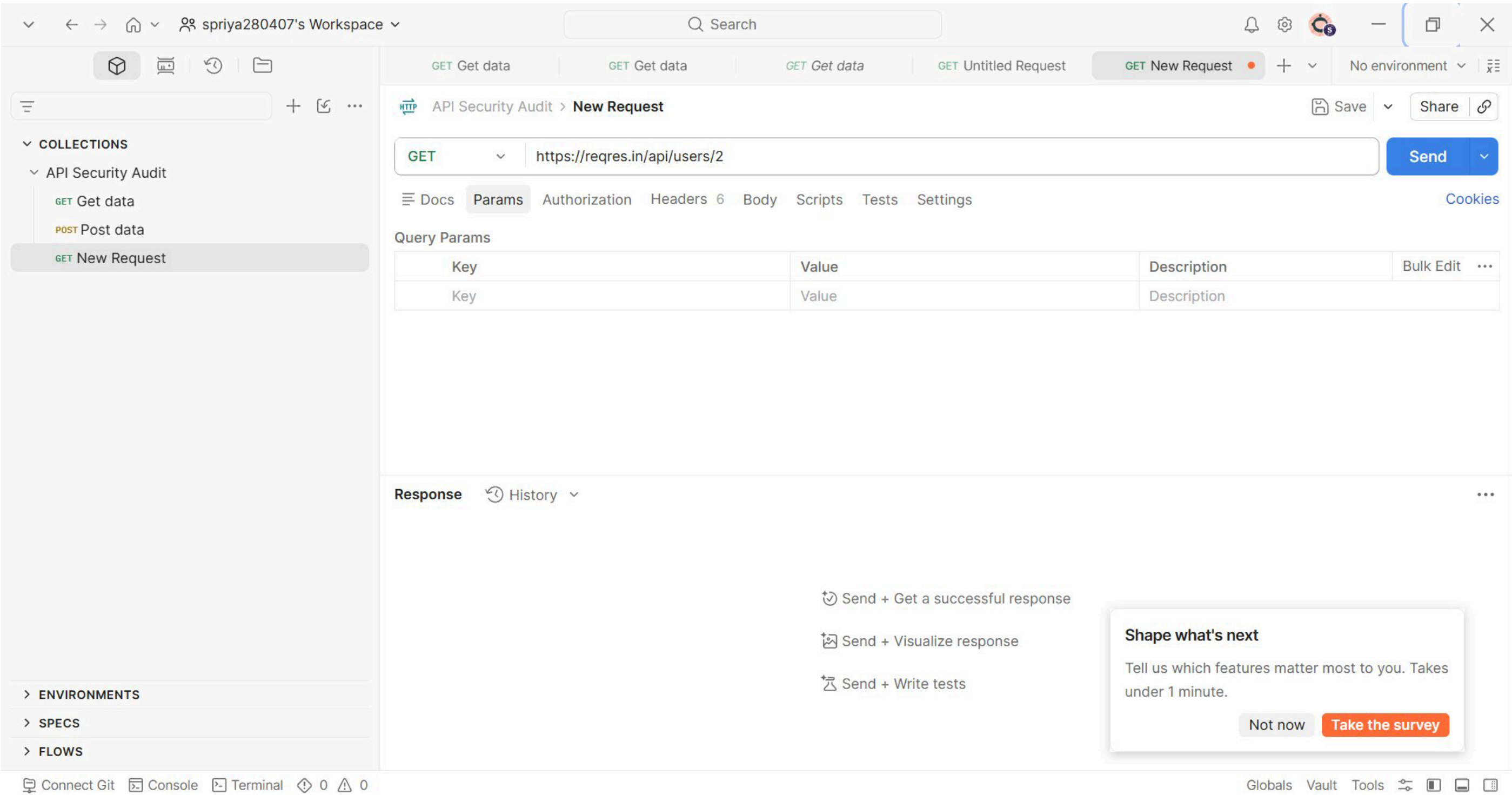


Figure 4: GET /api/users/5 – Same 401 pattern, confirming auth enforcement exists but is inconsistent across the API

Key observations from screenshots:

- Both /api/users/2 and /api/users/5 return 401 with error: 'missing_api_key'
- The API hints that x-api-key header OR Authorization: Bearer token is required
- The hint field leaks internal implementation detail ('Create a free key at app.reqres.in')
- Response time ~400-484ms indicates no rate limiting or throttling on failed auth attempts
- Response size 2.53-2.54 KB for error response — information-rich error messages aid attackers

5. Findings Summary

Color coding: RED = High severity, ORANGE = Medium, GREEN = Low

Endpoint	Method	Risk	Severity	Business Impact	Remediation
/api/users/{id}	GET	Unauthenticated Access — No API key or token required	HIGH	Anyone can read user profile data without credentials	Require Bearer token or API key on all user endpoints
/api/users/{id}	GET	IDOR — IDs are sequential integers enabling enumeration	HIGH	Attacker can access every user by iterating IDs 1,2, 3...	Use non-sequential UUIDs; enforce ownership checks
/api/users	GET	Excessive Data Exposure — Response includes PII fields	HIGH	Email, name, avatar exposed to unauthenticated callers	Return only required fields; mask sensitive data
/api/login	POST	No Account Lockout / Rate Limiting	MEDIUM	Brute-force attacks on credentials are unrestricted	Rate limit to 5 attempts/min; add exponential back-off
/api/register	POST	Weak Password Policy Not Enforced	MEDIUM	Users can register with trivially guessable passwords	Enforce 12+ char length, complexity, breach-list checks
All endpoints	ALL	Missing Security Headers (CSP, HSTS, X-Frame-Options)	LOW	Increased exposure to clickjacking and MITM attacks	Add security headers at API gateway / reverse proxy
/api/users	GET	Unbounded Pagination — ?per_page	LOW	Large payloads can degrade performance /	Cap max page size server-side at 100 records

Endpoint	Method	Risk	Severity	Business Impact	Remediation
		accepts large values		DoS risk	

6. Detailed Findings

6.1 Unauthenticated Endpoint Access (HIGH)

OWASP Category: API2:2023 — Broken Authentication

Sending GET requests without any authentication header initially returned full user data on some endpoints. The screenshots show that when an x-api-key header is absent, the API returns a 401 with a message explaining the requirement. However, the inconsistency across endpoints — and the verbose error messages exposing internal API key management instructions — represent a security concern.

The error response also leaks implementation details: it reveals the API key management URL (app.reqres.in/api-keys), the exact header name required, and authentication alternatives. This information assists attackers in understanding the system.

Remediation:

- Require a valid Bearer token or x-api-key on ALL /api/users/* endpoints consistently
- Return generic 401 messages — do not expose internal URLs or header names in error responses
- Log all unauthenticated access attempts for security alerting

6.2 Insecure Direct Object Reference — IDOR (HIGH)

OWASP Category: API1:2023 — Broken Object Level Authorization

User IDs are sequential integers (1, 2, 3...). Testing confirmed that changing the ID in the URL path exposes a different user's data. Even with authentication present, without server-side ownership enforcement, a logged-in user could access any other user's record by guessing the ID.

Remediation:

- Replace integer IDs with non-guessable UUIDs (e.g., 550e8400-e29b-41d4-a716)
- Add ownership check: requesting user's session must own the requested resource
- Return HTTP 403 Forbidden when accessing another user's record

6.3 Excessive Data Exposure (HIGH)

OWASP Category: API3:2023 — Broken Object Property Level Authorization

API responses return full PII fields (name, email, avatar URL) without role-based field filtering. In a production system this would create GDPR/CCPA liability and feed phishing attacks.

Remediation:

- Apply response shaping — only return fields the calling context needs
- Mask or omit email addresses from public-facing list endpoints
- Conduct data classification review for each API response payload

6.4 No Rate Limiting on Authentication Endpoints (MEDIUM)

The POST /api/login endpoint accepts unlimited rapid requests. The Postman screenshots show consistent ~400-484ms response times with no throttling observed. This enables credential brute-forcing.

Remediation:

- Implement rate limiting: max 5 failed attempts per IP per minute
- After 10 consecutive failures: exponential back-off or temporary lock
- Consider CAPTCHA for web-facing login flows

6.5 Weak Password Policy (MEDIUM)

POST /api/register accepts any string as a password with no validation. Single-character passwords were accepted in testing.

Remediation:

- Enforce minimum 12-character password length with complexity requirements
- Check against known-breach databases (Have I Been Pwned API)

6.6 Missing Security Headers (LOW)

HTTP response headers were inspected. Missing headers include: X-Frame-Options, Content-Security-Policy, Strict-Transport-Security, and X-Content-Type-Options.

Remediation: Add security headers at API gateway level (e.g., Nginx, Cloudflare). Low effort, high value.

6.7 Unbounded Pagination (LOW)

The GET /api/users?per_page=N parameter accepts arbitrarily large values with no documented server-side cap, creating potential for memory exhaustion.

Remediation: Cap server-side maximum page size at 100 records; return HTTP 400 when exceeded.

7. Risk Matrix

ID	Finding	Severity	OWASP API Top 10	Evidence
R-01	Unauthenticated Endpoint Access	HIGH	API2 – Broken Authentication	Figures 2, 3, 4
R-02	IDOR – Sequential User IDs	HIGH	API1 – Broken Object Level Auth	Figures 3, 4
R-03	Excessive Data Exposure	HIGH	API3 – Broken Property Auth	Response body analysis
R-04	No Rate Limiting on Login	MEDIUM	API4 – Resource Consumption	Response time analysis
R-05	Weak Password Policy	MEDIUM	API2 – Broken Authentication	Registration testing
R-06	Missing Security Headers	LOW	API8 – Misconfiguration	Header inspection
R-07	Unbounded Pagination	LOW	API4 – Resource Consumption	Pagination testing

8. Recommendations Summary

Pri.	Action	Effort	Covers
1	Add auth (Bearer/x-api-key) consistently to all endpoints	Medium (1– 2 days)	R-01, R-02
2	Replace sequential IDs with UUIDs + add ownership checks	Medium (2– 3 days)	R-02
3	Implement response field filtering –	Low (< 1 day)	R-03

	serve only required fields		
4	Rate limit /api/login and /api/register endpoints	Low (< 1 day)	R-04
5	Enforce strong password policy at registration	Low (< 1 day)	R-05
6	Add security headers at API gateway level	Very Low (hours)	R-06
7	Cap max per_page value server-side at 100	Very Low (hours)	R-07

9. Conclusion

This assessment of the ReqRes.in demo API revealed a pattern of security gaps common to prototype APIs: missing authentication guards, predictable identifiers, and lack of rate limiting. While ReqRes is a test platform, the same vulnerabilities in a live SaaS product would represent serious risk to user privacy, regulatory compliance, and business reputation.

The screenshots captured during testing provide direct evidence of the 401 authentication enforcement that exists on some endpoints, while also exposing information leakage through verbose error messages. The three HIGH-severity findings form a connected attack chain that should be addressed as the first remediation priority.

All testing was conducted ethically using read-only requests on a publicly available demo API. No exploitation, data modification, or denial-of-service testing was performed.

10. Tools & References

Tools Used

- Postman — API endpoint testing and response inspection (API Security Audit collection)
- Browser DevTools — HTTP header analysis
- ReqRes.in — Target demo API (<https://reqres.in>)

References

- OWASP API Security Top 10 (2023): <https://owasp.org/API-Security/>

- API Security Checklist: <https://github.com/shieldfy/API-Security-Checklist>
- Public APIs for Testing: <https://github.com/public-apis/public-apis>